Rebuilding the project on Moty's own run

The project was measured on five invented years of about 5,600 invoices each. Those years are not Moty's operation and have been dropped. Everything below moves the project onto the one run that exists.

Read this whole page before touching a file.

Rules

1. **The tests are written already, and they fail.** You make them pass. You write no tests of your own.
2. **Never edit a test.** If a test looks wrong, stop and say why.
3. **Never weaken a threshold, skip a test, or mark anything xfail.**
4. **Never touch anything on the protected list** except where a step names the function.
5. **Run the whole suite between steps**, not just the new file.
6. **Nothing runs longer than ten minutes.** If it would, stop it, write down the cost, and carry on.
7. Comments say **why**, never what. Short sentences, plain words.
8. **Do not run the model, the loop or any training.** This pass builds the structure and makes the tests pass. Nothing is measured and nothing is published.

The data

One source, and only one:

```
01-invoice-pipeline/Invoices/live/
├── ledger.json          every document issued: 1,044
├── labels.csv           the answer key: 37 errors, 48 false positives
├── data/                what the reader got out of the PDFs: 1,003 rows
├── 2026-01/ .. 2026-10/  the PDFs themselves
└── erp/                 vendors, changes, orders, lines, contracts,
                         receipts, shipments, rate card, tariffs,
                         items, ptax, approval policy
```

The earlier draft of this page said there was an `invoices.csv` at the root. There is not. 1,044 documents were issued and 1,003 rows came back out of the PDFs, so **41 invoices were never read into a row** — the reader flagged or lost them. A model only ever sees the 1,003.

It runs from 2 January to 14 October 2026. The five `year-seed*` folders and `drift-seed7` are no longer used by anything.

Protected: do not rewrite

File	Why
`src/features/build.py`	the feature definitions are correct; only **add** `cut()`
`src/evaluation/splits.py`	`known_labels` and `week_of` are correct; only **add** what step 2 names
`src/models/rules.py`	the twelve checks and their learnt weights
`src/models/feedback.py`	the verdict becomes a label
`src/models/scoring.py`	the dashboard rows
`src/runs/loop.py`	the week-by-week walk
everything under `tests/`	written by the reviewer
`../03-payables-panel/`	**not part of this pass at all**

If a fix seems to need one of these rewritten, that is a finding. Write it down and stop.

Step 1. Load Moty's run

Test: `tests/data/test_live_run.py`

Add `load.live(root)` to `src/data/load.py`, beside the existing `read()`.

It takes the folder, reads `invoices.csv`, `invoice_lines.csv`, `labels.csv` and every CSV under `erp/`, and returns the same `Dataset` the rest of the code already expects. Dates come back as dates. `is_anomaly` is 1 where `anomaly` is filled. `found_after_days` is a number, or missing where there is no error. A folder with no `labels.csv` raises `FileNotFoundError`.

Done when the file passes, including the two tests that read the real run and check the four numbers the README publishes.

Step 2. Split by time

Test: `tests/evaluation/test_time_split.py`

Add to `src/evaluation/splits.py`:

- `by_time(table, share=None, cutoff=None)` returns `(train, test)`. Every training row is dated on or before every test row. Giving both `share` and `cutoff`, or neither, raises. A split that leaves no errors on the test side raises.
- `describe(table, share=None, cutoff=None)` returns the first date, the last date and the error count of each side, plus the cutoff.

Do not change `known_labels` or `week_of`.

Step 3. The ordinary measures

Test: `tests/evaluation/test_confusion.py`

Add to `src/evaluation/metrics.py`, keeping everything already there:

- `confusion(truth, called)` returning `caught`, `missed`, `false_alarm`, `left_alone`.
- `precision`, `recall`, `f1`. A figure that cannot be computed comes back as `nan`, never as zero.
- `recall_by_kind(truth, called, kinds)`, ignoring the empty kind.
- `pr_curve(truth, scores)` returning `precision`, `recall` and `average_precision`.

Step 4. The interval around every figure

Test: `tests/evaluation/test_intervals.py`

New file `src/evaluation/intervals.py`.

- `around(figure, truth, called, seed=0, level=0.95, draws=2000)` resamples the rows and returns `(low, high)`. The same seed gives the same interval. A figure that cannot be computed comes back as `(nan, nan)`.
- `separated(a, b)` is True only when the two intervals do not overlap at all. Touching at a point is not separated.
- `compare(name_a, a, name_b, b)` returns both intervals, whether they are separated, and the winner — `None` when they are not.

This module is the project's whole answer to having 37 errors. It decides nothing by itself; it stops a number being read as more than it is.

Step 5. Cut the feature set

Test: `tests/features/test_feature_cut.py`

Add `cut(table, events_per_feature=10, cutoff=None)` to `src/features/build.py`. Do not change the features themselves.

It returns the kept feature names, in `COLUMNS` order. It keeps about one feature per ten errors in the rows it is given. It drops features that never move. It is deterministic. It refuses rows dated after `cutoff`, and refuses a table with too few errors to buy a single feature.

Step 6. Say what is gone

Nothing is measured in this pass, so nothing is reported. What must happen is removal:

1. `src/data/load.py`: `DATASETS` and `available()` point at the `year-seed*` folders. Leave `read()` working, but nothing new may call it.
2. `run.py`: the `features`, `walk`, `compare`, `retrain`, `loop`, `score` and `metrics` commands all assume the invented years. **Do not rewrite them in this pass.** Add one line of help text to each saying it is being rebuilt on the live run.
3. `docs/removals.md`: add an entry for every place you found a `year-seed` or `drift-seed` reference still standing, with its file and line. Do not delete those references.

Step 7. The commands, on the real run

Tests: `tests/runs/test_commands.py`, `tests/test_readme.py`

Steps 1 to 6 built the parts. Nothing yet runs end to end, and the project is meant to be left running for days. `run.py` gets four commands, and every old one that names an invented run is removed.

Command	Does
`prepare`	loads the run, builds the features, cuts them, writes the table, reports rows, errors, features before and after, and the kept list
`train`	splits by time, fits each model and the clerk's checks, reports every figure with its interval, and says which side of the split each date fell on
`score`	writes one row per invoice where the dashboard reads it; safe to run again over its own output
`serve`	keeps the gauges up

Three things make it fit for days of running, and all three are tested:

1. **No command may name `year-seed` or `drift-seed`.** Remove the old `features`, `walk`, `compare`, `retrain` and `loop` entry points rather than patching them.
2. `score` **is safe to run twice.** The second run replaces the first, with no duplicated rows.
3. **Every command appends one JSON line to a log**, with a timezone aware timestamp and what it did. Days of running are only readable through that file.

`tests/test_readme.py` was rewritten: it now checks the counts on the front page against the answer key, that no result is printed while none has been measured, and that every command the page shows is one `run.py` dispatches. It fails until this step is done.

Step 8. What the dashboard is given

Test: `tests/models/test_published_figures.py`

Add `publish_report(report, path)` to `src/models/exporter.py`, keeping `publish()` as it is.

It writes one JSON file holding the split, the baseline, and every model's confusion matrix, precision, recall, F1, average precision, recall by kind, and errors caught as more invoices are checked — each figure with its interval. It also writes a verdict: whether the two intervals are separated, and the winner when they are, `null` when they are not.

That file is what the four panels are drawn from. **Do not draw anything.** Do not touch `../03-payables-panel/`.

Step 9. Finish the clear-out

Four things from the invented runs are still standing. None of them needs a test; they need deleting.

1. `data/features/year-seed1.csv` to `year-seed5.csv`, about 3.5 MB of cached features from runs that no longer exist. Delete the files.
2. `src/runs/loop.py`, `training_floor`, still defaults to the four year seeds. Step 10 rewrites this function; if you reach step 10 first, this is done there.
3. `tests/models/test_honesty.py` and `tests/runs/test_loop_story.py` read `year-seed1` and `drift-seed7`. Those datasets are gone, so the tests cannot run. **Move both files to `tests/_retired/`** with a one-line note at the top saying what they measured and why they stopped. Do not delete them and do not rewrite them.
4. `docs/removals.md` item 3 lists every remaining reference. Update it to what is true after this step.

Step 10. The command that is left running

Test: `tests/runs/test_watch_command.py`

`prepare`, `train` and `score` each do one thing once. Nothing keeps the project alive, and nothing exercises the part that matters: a verdict becomes a label, a fall starts a challenger, a better model takes over.

Add `watch` to `run.py`, and rewrite `src/runs/loop.py` to work on the live run instead of the invented years.

Per week, in order:

1. Score the week with the model on duty. Never with one that saw it.

2. Build the verdicts as of the week's end, turn them into labels.
3. Watch the precision and the drift.
4. On a fall or a drift, train a challenger on the verdicts, and record which of the two triggered it.
5. Judge the challenger on the weeks since it was trained, with the intervals from step 4. **It takes over only when the intervals are separated.** With 35 errors that will rarely happen, and a promotion on overlapping intervals is the mistake this project exists to avoid.
6. Append the scored rows and one JSON line to the log.

It must survive being stopped: `watch` keeps its place in a state file, and starting again carries on from the next week without redoing or losing one.

`training_floor` loses its `year-seed` default here.

Step 11. Publish the result, whoever won

Test: `tests/test_published_result.py`

A report now exists at `data/model-report.json`, and it says the clerk's twelve checks beat every model on the held-back weeks, with every interval overlapping. **That is the result.** Write it into the README's Results section, replacing "None yet".

What the section must carry:

- Each model, with its precision, recall and average precision, and each one's interval beside it.
- The verdict in words: the intervals overlap, so the comparison is not decided.
- How many errors are on the test side, and between which dates.
- If the clerk's checks come out ahead, that near the top, in plain words, not buried under the model rows.

Then rebuild the PDFs in `docs/`. They still carry the invented years.

Step 12. A retraining obeys the cut

Test: `tests/models/test_retrain_uses_the_cut.py`

`watch` retrains through `lifecycle.retrain`, which fits all 24 features. `prepare` cuts the set to about three, because 35 errors buy three. So the model on duty after a retraining is the overfitted one, and every figure printed beside it measures a different model.

Change `src/models/lifecycle.py`:

- `retrain(table, as_of, feedback, seed=0, features=None)` fits only the named features, and raises when none are named. No default.

- `challenge(model, held_out, features=None)` scores on the same features, and raises when they differ from the model's own.
- `features_of(model)` returns the list the model was fitted on.

Then pass the kept list from `prepare` through `watch` into both.

Two corrections to earlier steps

Rule 4 was impossible, and that is my fault. It said no file may name `year-seed` or `drift-seed`. Two of my own tests name those strings in a docstring and in an assertion, precisely to forbid them in production code. The rule should have read: **no file under `src/` or `run.py` may name them.** Measured against that, the clear-out is done.

`test_readme.py` **and** `test_published_result.py` **contradicted each other.** One required "None yet" in the Results section; the other forbade it. `test_readme.py` has been rewritten: it now requires "None yet" only while `data/model-report.json` is absent, and requires a printed figure once it is there.

Order of work

1 to 12, in order. Step 4 needs 3. Step 7 needs 1 to 5. Step 8 needs

1. Step 10 needs 7 and finishes step 9's second item. Step 11 needs a report on disk, which step 7 already produced.

One thing to know before step 10

Everything in `data/model-report.json` and `data/runs.log` today was written by the test suite, into pytest's temporary folders. **Nothing has been run for real yet.** Step 10 is the first time the project runs on its own paths.

The counts, corrected

The answer key holds 1,044 rows, but only **1,003 invoice numbers**: 41 numbers were issued twice, revised or resent. Counted over distinct numbers the run holds **1,003 invoices, 35 errors and 46 false positives**. Those are the numbers on the README, and the ones the tests check.

When you are done

Say so, and report:

- the number of tests passing, and how long the suite took
- every protected file you touched, and which named function
- anything you found that the steps did not cover

When you are stuck

Stop and write what you tried. Do not edit a test, do not weaken a threshold, do not touch the protected list, and do not run the model. A step that cannot be done as written is a fault in the step, and the step gets rewritten, not the code.